

Introduction to Reinforcement Learning

6/10

Jean Martinet

MSc DSAI

2024 – 2025

- Introduction
 - Course 1 : Introduction to Reinforcement Learning (RL)
- Part I on tabular methods
 - Course 2 : Markov Decision Processes
 - Course 3 : Dynamic programming in RL
 - Course 4 : Temporal difference 1/2 (Q-learning)
 - Course 5 : Temporal difference 2/2 (SARSA)
- Part II on approximate methods
 - **Course 6 : Value function approximation**
 - Course 7 : Eligibility traces
 - Course 8 : Policy gradient 1/2 (REINFORCE)
 - Course 9 : Policy gradient 2/2 (actor-critic methods)
 - Course 10 : Projects presentation session

Today's menu

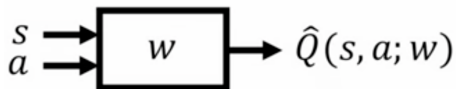
- Motivation
- Value Function Approximation
- ANN as a universal approximator

- DP and TD (and MC) are **tabular methods**
 - When we can discretise states and actions, limited number, write value as a table
- In real life, things are more complex
- Continuous/very large state and/or action spaces
 - Go game : 2^{170} states ($>$ nb atoms in universe!), 400 actions
 - Atari games : 240×160 dimensions
 - Robotics : multiple degrees of freedom, continuous action space
 - etc.

- In practice, most state will seldom be seen
- Two-fold problem
 - Memory needed to store huge tables
 - Convergence time to fill them (problem of *generalisation*)
- Idea : take examples from the value function, and attempts to generalise
 - *Function approximation is an instance of supervised learning*
- Assume that we have an oracle to tell us $V^\pi(s)$ for every s
 - We query any state, and we get the true value
 - We could make an (almost) comprehensive mapping $(s, V^\pi(s))$
- Linear function approximation vs non-linear (e.g. with an ANN)

Value Function Approximation

- The approximate value function is no more represented as a table
 - Parametrised function form $\hat{V}^\pi(s; w) \approx V^\pi(s)$
 - $w \in \mathbb{R}^d$ is a parameter vector



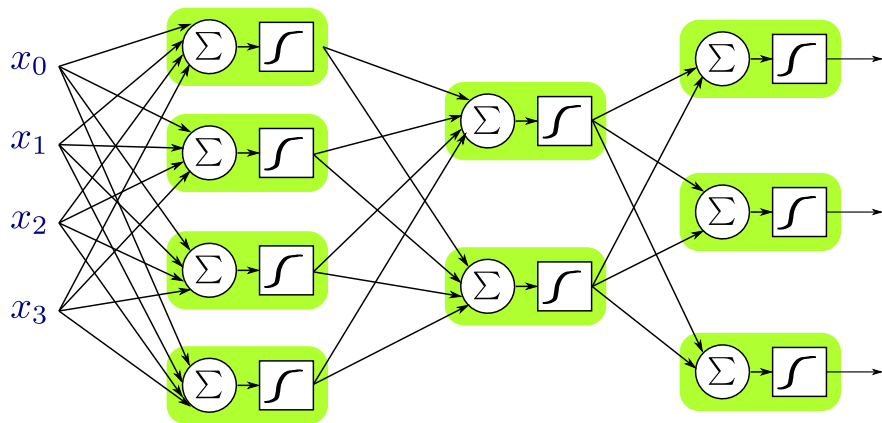
How to proceed ?

$$\hat{V}^{\pi}(s) = V_w(s) = V(s; w)$$

$$\hat{Q}^{\pi}(s, a) = V_w(s, a) = V(s, a; w)$$

- Use a parametrised function to estimate state value function and state-action value function
 - Compact representation that generalises across states and actions
 - Reduce memory, computation, experience (data) needed to learn
 - We do not want to store information for all state-action individual pairs
- Which approximator ?
 - Any can do (Decision Tree, Nearest Neighbour, Artificial Neural Networks, etc.)
 - Better if differentiable
 - Supervised machine learning is good for VFA

Multilayer feed-forward neural network as a universal approximator



- It is proven that (Cybenko and followers) :
 - all boolean functions can be represented using a single hidden layer
 - all bounded continuous functions can be represented using a single hidden layer, with an arbitrary precision
 - all functions can be represented using 2 hidden layers, with an arbitrary precision
- But :
 - How to determine the topology of the neural network ?
 - How many neurons per layer ?
 - How to tune the weights ? (learning phase)
 - (Assumption that you already know a lot about this!)

- All values are real values from $[0 \ 1]$.
- If not, scale the values : $x = \frac{u - u_{\min}}{u_{\max} - u_{\min}}$
- For discrete values, unary encoding form. As for example :

	x_1	x_2	x_3
flour	1	0	0
butter	0	1	0
sugar	0	0	1

- For a binary classification :
 - one output neuron
 - if $y < 0.5$: class 0
 - else class 1
- For a multi-class classification (n classes) :
 - one output neuron : divide $[0\ 1]$ by n
 - n output neurons
 - each neuron corresponds to probability over classes

1 Split data set (randomly) into three subsets :

- Training set : used for adjusting weights
- Validation set : used to stop training
- Test set : used to evaluate performance

2 Pick random, small weights as initial values

3 Perform iterative minimization of error over training set

4 Stop when error on validation set reaches a minimum (to avoid overfitting)

5 Repeat training (from step 2) several times (to avoid local minima)

6 Use best weights to compute error on the test set.

If the data set is too small to be divided into 3 subsets, use cross-validation

Discussion : Credit Assignment Problem

- Feedback can be delayed
- A central question in RL
 - How do you distribute credit for success among the many decisions that may have been involved in producing it?
- Two types
 - Temporal
 - Structural – where the interaction is made with a system with many parts : which one deserves credit for a good score / which part to blame for a bad score?
- Backprop is structural CAP : assign credits to the weights of the NN

- From humans? expert? the world?
- Ex : learn to predict tomorrows' temperature
 - Input = today / label = tomorrow
 - Error = (label - output), that is :
 - (tomorrow's temp - today's prediction of tomorrows' temp)
 - "self-supervised" : no external supervisor, just observations of the environment

Discussion : Origin of labels

- Idea of Rich Sutton (PhD student of Andy Barto in the 80's) : predict your own predictions!
 - Input = today
 - Label = (tomorrow + tomorrow's prediction of day after tomorrow)
 - Error = (tomorrow's temp + tomorrow's prediction - today)
- This is TD error

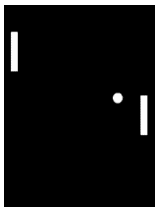
$$(r_{t+1} + V_{t+1}) - V_t$$

$$r_{t+1} + (V_{t+1} - V_t)$$

- Neuroscience perspective : signal is TD error
- Backprop as CAP back propagation
 - In RL, the unit follows the gradient, not with BP but with statistical estimate
 - No one uses RL to train DNN, it takes much longer, BP is faster
 - But RL does not need labels!
 - And it is bio-plausible (no labels in the brain)

Neural network with tensorflow.keras library and python3

- installation : `pip3 install --upgrade pip ; pip3 install tensorflow`
- choosing a layer type
 - fully connected or dense
 - convolutional : `conv`
- choosing the activation function
 - hidden layer : sigmoid, tanh ($\tanh(t) = 2\text{sig}(2t) - 1$), relu
 - output layer : linear (regression) or softmax (classification)
- choosing a loss function
 - `categorical_crossentropy` for multiclass classification
 - `binary_crossentropy` for binary classification
 - `mse` for regression
- choosing a learning rate
 - fixed : not too small, not too large. Could be $\eta = 0.01$
 - adaptive : Momentum, Adagrad, RMSProp, Adam, ...
- ways of creating a simple network :
 - `sequential` model
 - functional API
 - subclassing



- With the Atari game Pong :
 - Record a game where you (human player) play well
 - Generate a dataset of (X,y) where the input X is an image and the desired output y is the right action to take
 - Train an ANN with this data and use this estimator as an oracle

- Here, we use an ANN to directly estimate actions given a state
- Procedure
 - Record a game where you play well (a few hundreds frames)
 - Generate a dataset of (X,y) where input X is a meaningful representation of state, and desired output y is the right action to take
 - ✓ • Train an ANN with this data and use this estimator as an oracle
 - Works better if X includes a wide variety of different situations
 - Vary the topology of the network (number of layers, number of neurones per layer)
 - Check the performance of the ANN (accuracy on test data) BEFORE using it in the agent